

SYSTEM ARCHITECTURE

CineMatch

A conversational movie-recommendation chatbot — FastAPI backend, LLM-driven dialogue, and real-time streaming-availability verification.

Stack: FastAPI (Python 3.12) · Azure OpenAI Responses API · TMDB · Watchmode · OMDb · optional Redis

Frontend: Server-rendered Jinja2 + vanilla JS (no build step)

Repository: github.com/ananya2104/Cinematch

Generated: 2026-08-09

1. Overview

CineMatch is a single FastAPI deployable that runs a multi-turn chat conversation to establish what the user wants to watch, then asks Azure OpenAI to *curate* real movie titles from its own knowledge (not a database query), and verifies each candidate for real before showing it — confirming it exists on TMDB, that it is genuinely streaming on one of the user's selected platforms, and resolving a working deep link and IMDb rating. Suggestions that don't check out are silently dropped rather than shown broken.

There is no message queue, no microservice split, and no build pipeline — one Python process serves the API, the HTML template, and the static assets; conversation state lives in an in-memory store by default and swaps transparently to Redis when REDIS_URL is set.

2. Runtime components

Every request enters through `app/main.py`, which owns session state and decides what kind of turn this is; it delegates language/extraction work to `llm.py` and availability verification to `recommender.py`, which in turn fans out to three independent third-party data providers.

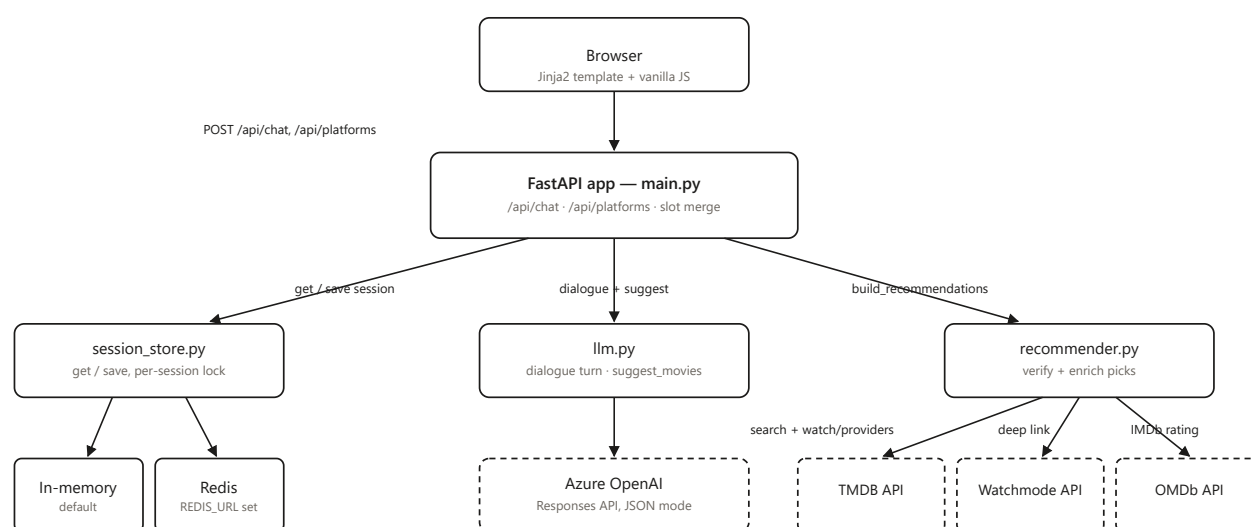


Figure 1. Call graph for one request: `main.py` is the only module that knows about HTTP; it hands off to `llm.py` for language/dialogue and `recommender.py` for verification, which is the only module that talks to the three external data providers. Dashed boxes are third-party services outside this codebase.

3. Module responsibilities

```

app/
├─ main.py          (299 ln) FastAPI app + routes; owns session lifecycle,
│                    slot merging (_merge_slots), and the branching that decides
│                    whether a turn asks a question, shows the platform
│                    checklist, or triggers a recommendation build
├─ llm.py           (334 ln) Azure OpenAI wrapper – structured-JSON dialogue turns
│                    (run_dialogue_turn) and movie curation (suggest_movies)
├─ recommender.py   (212 ln) Verifies each LLM-suggested title for real: TMDB lookup,
│                    watch-provider check, deep-link + IMDb-rating enrichment;
│                    owns the platform catalog and the retry-round loop
├─ tmdb.py          ( 71 ln) TMDB client – search a title, fetch watch/providers
├─ watchmode.py     ( 69 ln) Watchmode client – deep link to the exact platform
├─ omdb.py          ( 31 ln) OMDb client – real IMDb rating by IMDb id (optional)
├─ http_client.py   ( 91 ln) Shared GET-JSON helper – httpx first, falls back to
│                    shelling out to curl on a transport-level failure
├─ session_store.py (108 ln) SessionStore interface + two implementations
│                    (InMemorySessionStore, RedisSessionStore) – same shape, chosen
│                    once at startup based on REDIS_URL
├─ models.py        ( 94 ln) Pydantic models: Slots, SessionState, request/response
├─ config.py        ( 40 ln) Settings loaded from .env (Azure/TMDB/Watchmode/OMDb keys,
│                    region, CORS origins, optional Redis URL)
├─ templates/index.html    Chat UI markup (Jinja2 – served when the app hosts
│                    its own frontend)
├─ static/css, static/js    Dark-theme styling; chat logic, i18n, localStorage
│                    session persistence, rendering

docs/                    Static export of the frontend (same app.js/style.css,
                        de-templated index.html) for hosting separately on
                        GitHub Pages – see §6

```

4. Data model

Everything the app knows about a conversation lives in one `SessionState`, keyed by a client-generated `session_id` the frontend keeps in `localStorage`.

Model	Purpose	Key fields
Slots	Everything extracted from the conversation so far — accumulates across turns, mutated in place by <code>_merge_slots</code>	<code>basis</code> , <code>mood</code> , <code>genres[]</code> , <code>duration_max_minutes</code> , <code>industry</code> , <code>company</code> , <code>recency</code> , <code>era</code> , <code>min_rating</code> , <code>topic</code> , <code>actor</code> , <code>director</code> , <code>platforms[]</code>
<code>SessionState</code>	The full server-side session record — one per <code>session_id</code>	<code>history[]</code> , <code>slots</code> , <code>language</code> , <code>turn_count</code> , <code>recommended</code> , <code>platforms_resolved</code> , <code>recommended_ids[]</code> , <code>recommended_titles[]</code>
<code>DialogueTurnResult</code>	Structured JSON the LLM returns for <i>every</i> turn — a delta on top of existing slots, never a full recap	<code>reply_text</code> , <code>detected_language</code> , <code><slot fields></code> , <code>ready_to_recommend</code> , <code>needs_platform_selection</code> , <code>wants_recommendations</code> , <code>quick_replies[]</code>
<code>Recommendation</code>	One verified, streamable movie — the only shape the frontend ever renders as a card	<code>tmdb_id</code> , <code>title</code> , <code>year</code> , <code>poster_url</code> , <code>platform</code> , <code>blurb</code> , <code>watch_url</code> , <code>imdb_rating</code>

Delta merge, not recap. The dialogue prompt tells the model to only fill in what the latest message actually changes and leave the rest null. Naming a new actor, director, or topic is treated as a hard pivot in `_merge_slots` — it clears mood/genre/ duration/industry/recency/era/rating from earlier in the conversation, since a stale "short comedy" filter combined with "directed by Nolan" would otherwise return nothing.

5. Anatomy of one /api/chat request

`_chat_impl` in `main.py` routes every turn down one of two paths, depending on whether this session has ever produced a recommendation yet.

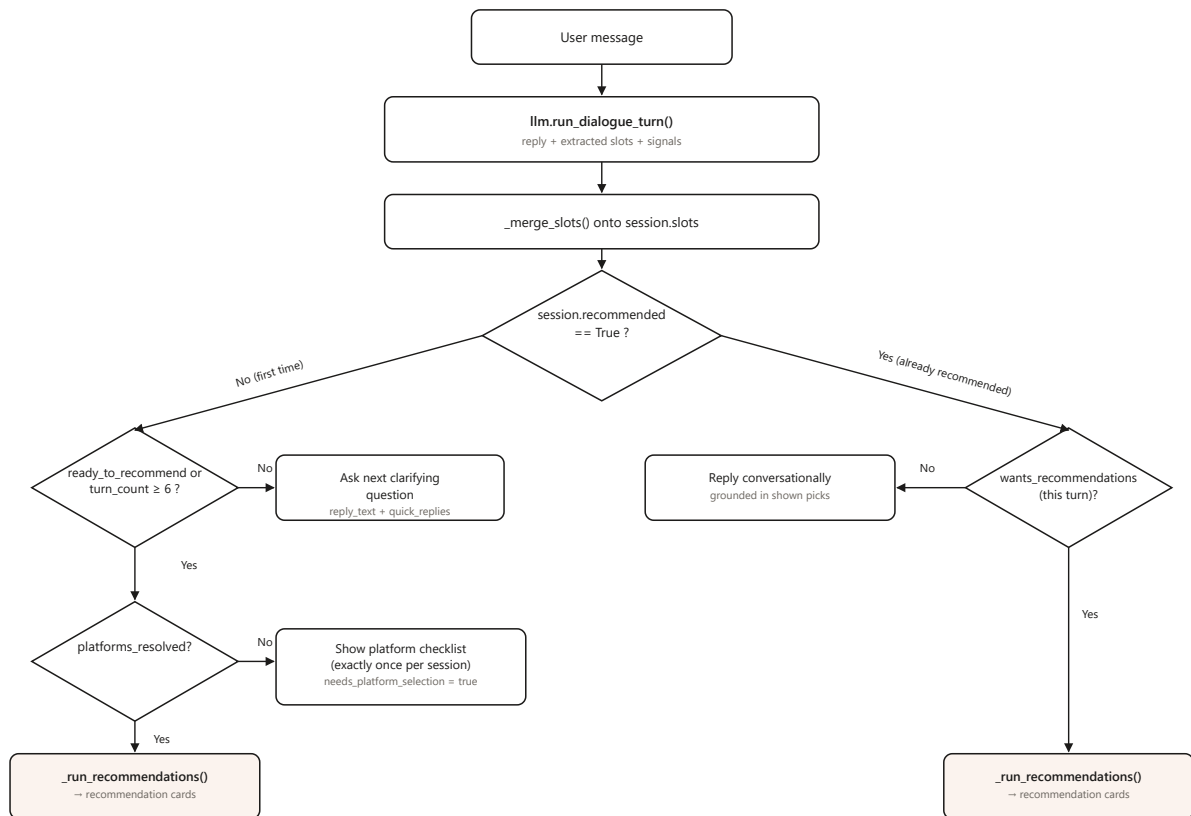


Figure 2. Every request runs the same dialogue turn and slot merge, then splits on `session.recommended`: a first-time session works through clarifying questions and a one-time platform checklist before it ever recommends; once recommended, later turns either fetch another batch or reply conversationally, decided by the model's own `wants_recommendations` signal rather than by keyword matching.

6. Recommendation verification pipeline

`recommender.build_recommendations` is the only place the app talks to TMDB, Watchmode, and OMDb. It treats the LLM as a curator, not a database: the model proposes real titles from memory, and every proposal is independently checked before it can reach the user.

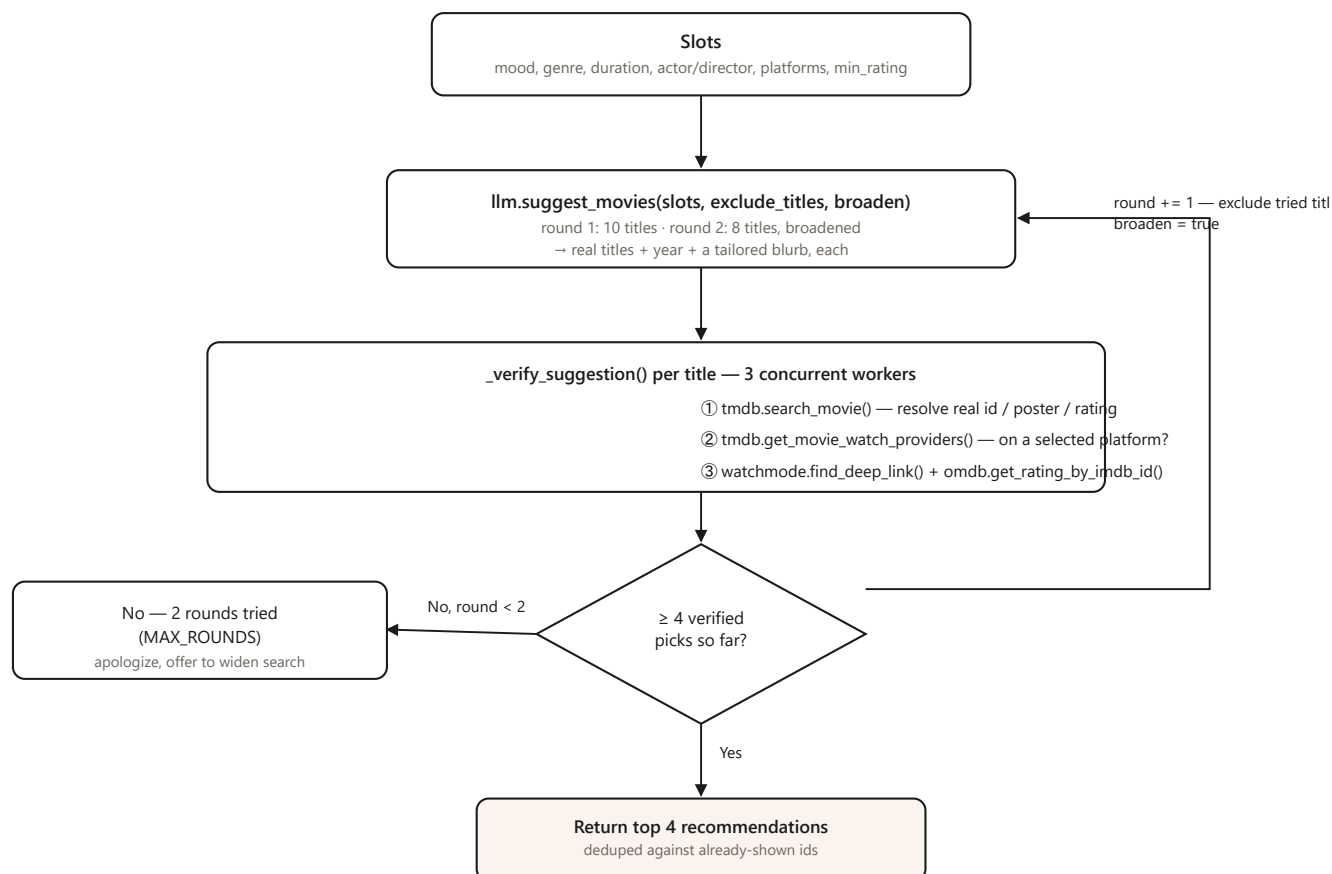


Figure 3. Verification runs three candidates at a time (`VERIFY_WORKERS = 3` — deliberately modest, since some networks reset a TCP connection outright once too many land on one host at once; see `http_client.py`). Anything that fails any check is dropped, never shown as a broken card; if fewer than four picks survive, a second round asks the model again with the failed titles excluded and instructions to prioritize availability over an exact thematic match.

7. Resilience & session storage

Networking fallback

`http_client.get_json` tries `httpx` first; on a transport-level failure it flips a process-wide flag and shells out to `curl` for every subsequent call, since the two use different TLS stacks and one worked where the other was reset mid-handshake on the original dev network. Retries with backoff; 4xx responses are never retried.

Every session read/write for a given `session_id` is serialized through `SessionStore.session_lock` (a real distributed lock in the Redis case), so two requests for the same session — a double-click, a race between tabs — can't clobber each other's state.

Session storage

`SessionStore` is one interface with two implementations, chosen once at startup by whether `REDIS_URL` is set — nothing else in the app changes based on which is active. In-memory is per-process and TTL-evicted; Redis survives restarts and is correct across multiple app instances. If Redis is configured but unreachable at boot, the app logs a warning and falls back to in-memory rather than failing to start.

8. Deployment topology

The current deployment splits frontend and backend across two hosts, talking over CORS; local development instead serves both from one Uvicorn process with no CORS involved at all.

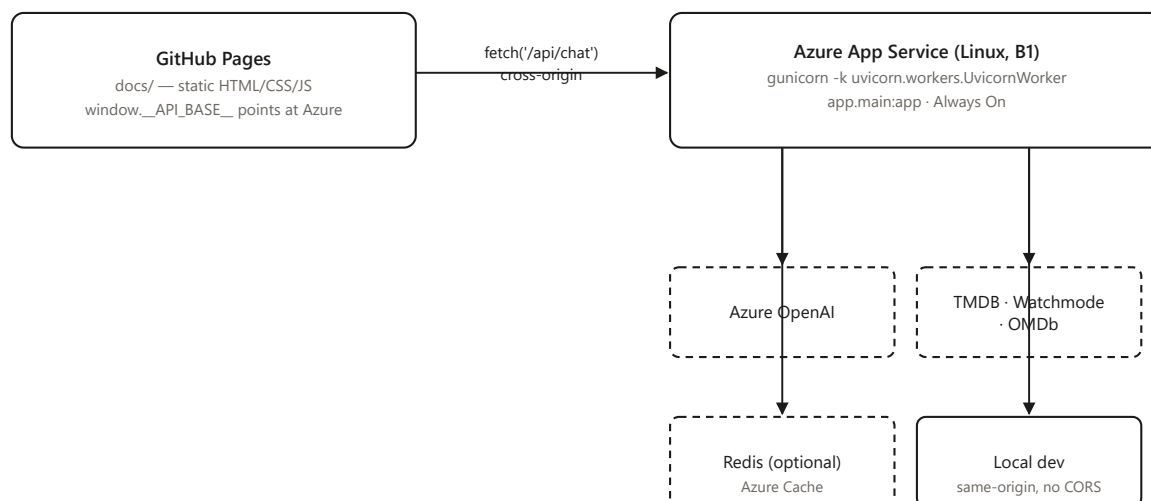


Figure 4. The two ends of the CORS connection are `CORS_ALLOWED_ORIGINS` in `app/config.py` and `window.__API_BASE__` in `docs/index.html`. Locally, `app/templates/` + `app/static/` are served by FastAPI itself — same host, no CORS, and unaffected by the split-hosting setup.

9. Known limitations

- **Streaming catalog is curated to India** (`STREAMING_REGION=IN`) and six platforms that exist in TMDB's India catalog today.
- **No rate limiting yet** on `/api/chat` — nothing currently stops a script from hammering it and burning through the Azure OpenAI/TMDB/OMDb quotas.
- **Replies arrive all at once**, not streamed — the Responses API supports SSE, which would make the multi-second recommendation build feel more responsive.
- **Only in-process logging** — no structured telemetry (e.g. Application Insights) yet to see real usage patterns and error rates.
- **No persistent accounts** — Redis (when enabled) is for surviving restarts/scaling within a session's lifetime, not for a returning user's history across days/devices.